# UTS

# PEMBELAJARAN MESIN




NAMA : HANA MALIDHA HASAN

NIM : 202431057

KELAS : E

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC : 9

ASISTEN : 1. Muhammad Caisar Gemilang Pratama

2. Benedict Samuel Ardy

## INSTITUT TEKNOLOGI PLN

## TEKNIK INFORMATIKA

## 2026/2027

# DAFTAR ISI

**DAFTAR ISI** ..... **ii**
**BAB I PENDAHULUAN** ..... **1**
1.1 Rumusan Masalah ..... 1
1.2 Tujuan Masalah ..... 1
1.3 Manfaat Masalah ..... 1
**BAB II LANDASAN TEORI** ..... **2**
2.1 Pembelajaran Mesin Terbimbing untuk Prediksi Diabetes ..... 2
2.2 Exploratory Data Analysis dan Preprocessing ..... 2
2.3 Regresi Linear ..... 3
2.4 K-Nearest Neighbors (KNN) ..... 4
2.5 Metrik Evaluasi ..... 4
**BAB III HASIL** ..... **6**
3.1 Gambaran Umum Praktikum ..... 6
3.2 Penjelasan Setiap Cell Kode dan Output ..... 6
3.3 Pembahasan Hasil ..... 24
**BAB IV PENUTUP** ..... **26**
Kesimpulan ..... 26
Saran ..... 26
**DAFTAR PUSTAKA** ..... **27**

# BAB I
# PENDAHULUAN

## 1.1 Rumusan Masalah

Berdasarkan dataset diabetes yang diberikan pada Project UTS Pembelajaran Mesin, rumusan masalah yang diangkat dalam laporan ini adalah sebagai berikut.

1. Bagaimana karakteristik dataset diabetes (100.000 baris dan 9 kolom) berdasarkan hasil *Exploratory Data Analysis* (EDA)?
2. Bagaimana tahapan *preprocessing* yang diperlukan (penanganan duplikat, *encoding* fitur kategorikal, standardisasi, serta pembagian data latih dan uji) agar data dapat dilatih menggunakan algoritma Regresi Linear dan *K-Nearest Neighbors* (KNN)?
3. Berapa akurasi dan metrik evaluasi lain yang dihasilkan oleh Regresi Linear dan KNN dalam memprediksi status diabetes?
4. Algoritma manakah yang lebih sesuai untuk dataset ini, dan faktor apa yang menyebabkan perbedaan kinerjanya?

## 1.2 Tujuan Masalah

Tujuan yang ingin dicapai dari praktikum UTS ini adalah:

1. Memahami struktur, distribusi, hubungan antarvariabel, dan potensi masalah kualitas data pada dataset diabetes melalui EDA.
2. Menerapkan *preprocessing* yang sesuai dengan kebutuhan algoritma Regresi Linear dan KNN.
3. Melatih model Regresi Linear dan KNN, melakukan *hyperparameter tuning* pada KNN, serta memperoleh akurasi hasil pengolahan data.
4. Membandingkan kedua model secara adil menggunakan akurasi, *precision*, *recall*, *F1-score*, dan ROC-AUC, kemudian menarik kesimpulan berdasarkan teori dan data pendukung.

## 1.3 Manfaat Masalah

Manfaat yang diperoleh dari praktikum UTS ini adalah:

1. Mahasiswa memperoleh pengalaman langsung dalam alur kerja pembelajaran mesin terbimbing, mulai dari EDA, *preprocessing*, pelatihan, hingga evaluasi model.
2. Mahasiswa memahami bahwa akurasi saja tidak cukup untuk data yang tidak seimbang, sehingga perlu dilengkapi *precision*, *recall*, *F1-score*, dan ROC-AUC.
3. Hasil perbandingan Regresi Linear dan KNN dapat menjadi gambaran awal pemilihan algoritma *baseline* untuk prediksi risiko diabetes berbasis data klinis sederhana.

# BAB II
# LANDASAN TEORI

## 2.1 Pembelajaran Mesin Terbimbing untuk Prediksi Diabetes

Pembelajaran mesin terbimbing (*supervised learning*) mempelajari pemetaan dari sekumpulan fitur $X$ ke label $y$ menggunakan data yang sudah berlabel. Pada penelitian ini label berupa status diabetes (0 = tidak, 1 = ya), sehingga persoalan yang dihadapi secara alami merupakan klasifikasi biner. Fitur yang tersedia meliputi usia, jenis kelamin, riwayat hipertensi, riwayat penyakit jantung, riwayat merokok, indeks massa tubuh (BMI), kadar HbA1c, dan kadar glukosa darah. HbA1c mencerminkan rata-rata kadar glukosa darah dalam beberapa bulan terakhir, sedangkan glukosa darah menunjukkan kondisi saat pengukuran, sehingga keduanya wajar menjadi prediktor yang kuat.

Karakteristik umum data diabetes adalah distribusi kelas yang tidak seimbang, karena penderita biasanya hanya sebagian kecil dari seluruh sampel. Pada prediksi risiko diabetes dengan kelas tidak seimbang, penyesuaian ambang keputusan, pembobotan biaya, dan *resampling* dapat memperbaiki *F1-score* dan *g-mean* pada ketiga algoritma yang diuji (Sadeghi et al., 2022). Temuan tersebut mengingatkan bahwa ukuran kinerja pada data semacam ini tidak boleh bertumpu pada akurasi semata.

## 2.2 Exploratory Data Analysis dan Preprocessing

*Exploratory Data Analysis* (EDA) adalah tahap memahami data melalui statistik deskriptif, visualisasi distribusi, pemeriksaan *missing value*, duplikasi, *outlier*, serta korelasi antarvariabel sebelum model dilatih. EDA membantu menentukan keputusan *preprocessing*, misalnya apakah *outlier* perlu dibuang atau tetap dipertahankan karena merepresentasikan kondisi klinis yang nyata.

Tahap *preprocessing* pada penelitian ini mencakup empat hal. Pertama, penghapusan baris duplikat agar sampel yang sama tidak muncul di data latih sekaligus data uji, yang akan membuat akurasi tampak lebih tinggi dari kenyataan. Kedua, *one-hot encoding* untuk fitur kategorikal nominal agar dapat diproses algoritma numerik. Ketiga, pembagian data latih dan data uji secara *stratified* sehingga proporsi kelas tetap terjaga. Keempat, standardisasi fitur menggunakan *z-score*:

$$z = \frac{x - \mu}{\sigma}$$

dengan $\mu$ dan $\sigma$ adalah rata-rata dan simpangan baku yang dihitung hanya dari data latih untuk mencegah kebocoran data (*data leakage*). Standardisasi sangat penting bagi algoritma berbasis jarak seperti KNN karena fitur berskala besar (misalnya glukosa darah 80–300) akan mendominasi fitur berskala kecil (misalnya HbA1c 3,5–9). Pemilihan metode penskalaan (*Min-Max*, *Standard*, atau

*Robust*) dan metrik jarak dilaporkan berpengaruh terhadap kinerja klasifikasi KNN pada data medis (Arora et al., 2022).

**2.3 Regresi Linear**

Regresi Linear memodelkan variabel target kontinu sebagai kombinasi linear dari fitur:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

Koefisien $\beta$ diestimasi dengan metode *Ordinary Least Squares* (OLS), yaitu meminimalkan jumlah kuadrat galat antara nilai aktual dan prediksi:

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = (X^\top X)^{-1} X^\top \mathbf{y}$$

Model ini mengasumsikan hubungan linear antara fitur dan target, galat yang saling bebas, serta ragam galat yang konstan. Karena fitur telah distandardisasi, besar koefisien dapat dibandingkan antarfitur: koefisien yang lebih besar menunjukkan pengaruh yang lebih kuat terhadap keluaran model untuk setiap kenaikan satu simpangan baku fitur.

Pada bentuk paling sederhana dengan satu variabel bebas, persamaannya menjadi $Y = a + bX$, dengan $a$ sebagai konstanta (intersep) dan $b$ sebagai koefisien regresi (kemiringan garis). Selisih $\varepsilon = y - \hat{y}$ antara nilai aktual dan prediksi disebut residu, dan metode kuadrat terkecil memilih garis yang meminimalkan jumlah kuadrat residu tersebut. Kekuatan hubungan linear antara dua variabel diukur dengan koefisien korelasi *Pearson* ($r$), sedangkan kuadratnya, $r^2$, adalah koefisien determinasi yang menyatakan persentase variasi variabel terikat yang dijelaskan variabel bebas. Tingkat hubungan dikategorikan sebagai berikut: $r^2 < 0{,}25$ sangat lemah, 0,25–0,50 lemah, 0,50–0,75 sedang, 0,75–0,90 kuat, dan $\geq 0{,}90$ sangat kuat. Pada regresi berganda seperti penelitian ini, ukuran yang sepadan adalah $R^2$ dari seluruh fitur.

Regresi Linear pada dasarnya dirancang untuk target kontinu. Ketika target berupa biner 0/1, keluarannya dapat dibaca sebagai skor mirip probabilitas (*linear probability model*) yang diubah menjadi kelas dengan ambang tertentu, lazimnya 0,5. Pendekatan ini sederhana dan cepat, tetapi keluarannya tidak dibatasi pada rentang [0, 1] sehingga dapat bernilai negatif atau lebih dari satu. Karena itu kualitas modelnya diukur dengan dua cara: metrik regresi (MSE, RMSE, MAE, $R^2$) pada skor kontinu, dan metrik klasifikasi (akurasi dan turunannya) setelah skor diberi ambang.

**2.4 K-Nearest Neighbors (KNN)**

KNN adalah algoritma non-parametrik berbasis *instance* yang tidak membangun model eksplisit saat pelatihan (*lazy learning*). Untuk sebuah data baru, KNN menghitung jaraknya ke seluruh data latih, memilih *k* tetangga terdekat, lalu menentukan kelas melalui pemungutan suara. Jarak yang paling umum digunakan adalah jarak *Euclidean*, dan prediksinya dapat berbobot (*weights*) sehingga tetangga yang lebih dekat memiliki pengaruh lebih besar:

$$d(\mathbf{x}, \mathbf{z}) = \sqrt{\sum_{j=1}^{p} (x_j - z_j)^2}$$

$$\hat{y}(\mathbf{x}) = \arg\max_{c} \sum_{i \in N_k(\mathbf{x})} w_i \, \mathbb{1}(y_i = c)$$

Langkah kerja KNN adalah sebagai berikut: (1) menentukan nilai *k*; (2) menghitung jarak data uji terhadap seluruh data latih; (3) mengurutkan jarak dari yang terkecil; (4) mengambil *k* data terdekat; dan (5) menentukan kelas berdasarkan suara terbanyak (*majority voting*). Selain jarak *Euclidean*, dikenal pula jarak *Manhattan* (*city-block*), *Chebyshev*, dan *Minkowski* yang merupakan generalisasinya. Apabila hasil pemungutan suara seri, nilai *k* dikurangi satu atau dipilih *k* ganjil. Kelebihan KNN adalah sederhana, intuitif, cocok untuk data numerik, dan cukup tahan terhadap *noise* pada data latih; kekurangannya adalah lambat pada data besar karena jarak dihitung ke seluruh data latih, serta peka terhadap skala fitur dan nilai *k*.

KNN populer karena sederhana dan efektif, tetapi kinerjanya peka terhadap pemilihan nilai *k*, metrik jarak, fitur yang tidak relevan, serta ketidakseimbangan kelas, dan biaya komputasinya meningkat pada data besar (Halder et al., 2024). Nilai *k* yang terlalu kecil membuat model rentan terhadap *noise* (varians tinggi), sedangkan *k* yang terlalu besar membuat batas keputusan terlalu halus sehingga kelas minoritas tertutup oleh kelas mayoritas (bias tinggi). Pencarian *k*-tetangga terdekat yang tepat pada data berdimensi tinggi memerlukan perhitungan jarak yang mahal, sehingga mendorong penggunaan teknik pengindeksan dan pemartisian data untuk mempercepat pencarian (Ukey et al., 2023). Oleh karena itu pada penelitian ini nilai *k* dan *weights* dipilih secara sistematis menggunakan *GridSearchCV* dengan *stratified 5-fold cross-validation*, bukan ditentukan secara acak.

**2.5 Metrik Evaluasi**

Untuk keluaran kontinu Regresi Linear digunakan *Mean Squared Error* (MSE), *Root Mean Squared Error* (RMSE), *Mean Absolute Error* (MAE), dan koefisien determinasi $R^2$:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}, \quad \text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

$R^2$ dinilai lebih informatif dibandingkan SMAPE, MAE, MAPE, MSE, dan RMSE pada evaluasi regresi, karena nilainya dapat ditafsirkan tanpa bergantung pada skala data (Chicco et al., 2021). Pemilihan antara RMSE dan MAE bergantung pada distribusi galat, sehingga pelaporan kedua metrik secara bersamaan merupakan praktik yang wajar (Hodson, 2022). Pada penelitian ini keempat metrik dilaporkan.

Untuk keluaran kelas digunakan matriks konfusi (*TP, TN, FP, FN*) beserta akurasi, *precision*, *recall*, dan *F1-score*:

$$\text{Akurasi} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \cdot P \cdot R}{P + R}$$

Selain itu digunakan ROC-AUC yang menilai kemampuan model mengurutkan sampel positif di atas sampel negatif pada semua ambang keputusan, sehingga tidak bergantung pada satu ambang tertentu. Pada data yang tidak seimbang, akurasi dapat tampak tinggi hanya karena model memprediksi kelas mayoritas (Sadeghi et al., 2022). Maka akurasi dalam laporan ini selalu dibandingkan dengan akurasi *baseline* kelas mayoritas dan dilengkapi *recall* kelas diabetes.

# BAB III
# HASIL

## 3.1 Gambaran Umum Praktikum

Praktikum dikerjakan secara individu pada dataset 2 (*diabetes*) dengan algoritma Regresi Linear dan KNN, sesuai pembagian nomor 9 (NIM 202431057, Hana Malidha Hasan). Seluruh kode dijalankan di Python (pandas, scikit-learn, matplotlib, seaborn) dan disimpan pada berkas *202431057_UTS_PM.ipynb*. Alur pengerjaan terdiri dari 17 *cell* kode: *import* pustaka (Cell 1), pemahaman data (Cell 2–3), EDA (Cell 4–8), *preprocessing* (Cell 9–10), model Regresi Linear (Cell 11–12), model KNN (Cell 13–16), dan perbandingan model (Cell 17). Setiap *cell* pada bab ini ditampilkan beserta komentar NIM dan nama pada kode, outputnya, dan penjelasannya.

## 3.2 Penjelasan Setiap Cell Kode dan Output

### 3.2.1 Cell 1: Import library

In [1]:
```
#202431057_HANA MALIDHA HASAN
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, mean_squared_error, mean_absolute_error,
    r2_score, roc_auc_score, roc_curve, ConfusionMatrixDisplay)
import warnings; warnings.filterwarnings("ignore")

NIM = "202431057"
NAMA = "HANA MALIDHA HASAN"
SEED = 42
sns.set_style("whitegrid")
pd.set_option("display.width", 110); pd.set_option("display.max_columns", 20)


print("Library siap digunakan.", NIM, "-", NAMA)
```

*Gambar 3.1 Kode Cell 1 (watermark NIM)*

Out[1]:
```
Library siap digunakan. 202431057 - HANA MALIDHA HASAN
```

*Gambar 3.2 Output teks Cell 1*

**Kode.** Cell ini mengimpor pustaka yang dipakai sepanjang praktikum: *pandas* dan *numpy* untuk pengolahan data, *matplotlib* dan *seaborn* untuk visualisasi, serta modul *scikit-learn* untuk pembagian data, *GridSearchCV*, *StandardScaler*, *LinearRegression*, *KNeighborsClassifier*, dan berbagai metrik evaluasi. Variabel *NIM*, *NAMA*, dan *SEED = 42* didefinisikan agar identitas tercetak dan hasil dapat direproduksi. Setiap *cell* diawali komentar *#202431057_HANA MALIDHA HASAN* sebagai penanda NIM dan nama.

**Hasil.** Output berupa pesan konfirmasi bahwa pustaka berhasil dimuat beserta NIM dan nama. Tidak ada galat, sehingga lingkungan siap digunakan.

### 3.2.2 Cell 2: Memuat dataset

```
In [2]: #202431057_HANA MALIDHA HASAN
        df = pd.read_csv("02_diabetes.csv")
        print("Jumlah baris, kolom :", df.shape)
        print()
        df.info()
        df.head()
```

*Gambar 3.3 Kode Cell 2 (watermark NIM)*

```
Jumlah baris, kolom : (100000, 9)

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 9 columns):
 #   Column               Non-Null Count   Dtype
---  ------               --------------   -----
 0   gender               100000 non-null  object
 1   age                  100000 non-null  float64
 2   hypertension         100000 non-null  int64
 3   heart_disease        100000 non-null  int64
 4   smoking_history      100000 non-null  object
 5   bmi                  100000 non-null  float64
 6   HbA1c_level          100000 non-null  float64
 7   blood_glucose_level  100000 non-null  int64
 8   diabetes             100000 non-null  int64
dtypes: float64(3), int64(4), object(2)
memory usage: 6.9+ MB
```

Out[2]:

| | gender | age | hypertension | heart_disease | smoking_history | bmi | HbA1c_level | blood_glucose_level | diabetes |
|---|---|---|---|---|---|---|---|---|---|
| 0 | Female | 80.0 | 0 | 1 | never | 25.19 | 6.6 | 140 | 0 |
| 1 | Female | 54.0 | 0 | 0 | No Info | 27.32 | 6.6 | 80 | 0 |
| 2 | Male | 28.0 | 0 | 0 | never | 27.32 | 5.7 | 158 | 0 |
| 3 | Female | 36.0 | 0 | 0 | current | 23.45 | 5.0 | 155 | 0 |
| 4 | Male | 76.0 | 1 | 1 | current | 20.14 | 4.8 | 155 | 0 |

*Gambar 3.4 Output teks Cell 2*

**Kode.** Dataset *02_diabetes.csv* dibaca dengan *pd.read_csv*. Perintah *df.shape*, *df.info()*, dan *df.head()* dipakai untuk melihat ukuran, tipe data, dan contoh isi data.

**Hasil.** Dataset terdiri dari 100.000 baris dan 9 kolom, yaitu 8 fitur dan 1 target (*diabetes*). Tidak ada nilai *null* pada seluruh kolom. Terdapat dua kolom kategorikal (*gender* dan *smoking_history*), tiga kolom *float* (*age*, *bmi*, *HbA1c_level*), dan empat kolom *integer* (*hypertension*, *heart_disease*, *blood_glucose_level*, *diabetes*).

### 3.2.3 Cell 3: Statistik deskriptif, missing value, dan data duplikat

```
In [3]: #202431057_HANA MALIDHA HASAN
        print("Missing value per kolom:"); print(df.isna().sum().to_string())
        print("\nJumlah baris duplikat :", df.duplicated().sum())
        print("Kategori gender       :", df["gender"].unique().tolist())
        print("Kategori smoking      :", df["smoking_history"].unique().tolist())
        df.describe().T.round(2)
```

*Gambar 3.5 Kode Cell 3 (watermark NIM)*

```
Missing value per kolom:
gender                0
age                   0
hypertension          0
heart_disease         0
smoking_history       0
bmi                   0
HbA1c_level           0
blood_glucose_level   0
diabetes              0

Jumlah baris duplikat : 3854
Kategori gender       : ['Female', 'Male', 'Other']
Kategori smoking      : ['never', 'No Info', 'current', 'former', 'ever', 'not current']
```

Out[3]:

| | count | mean | std | min | 25% | 50% | 75% | max |
|---|---|---|---|---|---|---|---|---|
| **age** | 100000.0 | 41.89 | 22.52 | 0.08 | 24.00 | 43.00 | 60.00 | 80.00 |
| **hypertension** | 100000.0 | 0.07 | 0.26 | 0.00 | 0.00 | 0.00 | 0.00 | 1.00 |
| **heart_disease** | 100000.0 | 0.04 | 0.19 | 0.00 | 0.00 | 0.00 | 0.00 | 1.00 |
| **bmi** | 100000.0 | 27.32 | 6.64 | 10.01 | 23.63 | 27.32 | 29.58 | 95.69 |
| **HbA1c_level** | 100000.0 | 5.53 | 1.07 | 3.50 | 4.80 | 5.80 | 6.20 | 9.00 |
| **blood_glucose_level** | 100000.0 | 138.06 | 40.71 | 80.00 | 100.00 | 140.00 | 159.00 | 300.00 |
| **diabetes** | 100000.0 | 0.08 | 0.28 | 0.00 | 0.00 | 0.00 | 0.00 | 1.00 |

*Gambar 3.6 Output teks Cell 3*

**Kode.** Cell ini memeriksa *missing value* (*isna().sum()*), jumlah baris duplikat (*duplicated().sum()*), kategori unik pada kolom kategorikal, dan statistik deskriptif (*describe()*).

**Hasil.** Tidak ada *missing value*, tetapi ditemukan **3.854 baris duplikat** yang perlu ditangani. Kolom *gender* memiliki tiga kategori (*Female*, *Male*, *Other*) dan *smoking_history* memiliki enam kategori, termasuk *No Info* yang diperlakukan sebagai kategori tersendiri. Statistik deskriptif menunjukkan usia 0,08–80 tahun (rata-rata 41,89), BMI 10,01–95,69 (rata-rata 27,32), HbA1c 3,5–9,0 (rata-rata 5,53), dan glukosa darah 80–300 (rata-rata 138,06). Rata-rata *hypertension* 0,0749 dan *heart_disease* 0,0394 berarti 7,49% dan 3,94% pasien memiliki riwayat tersebut.

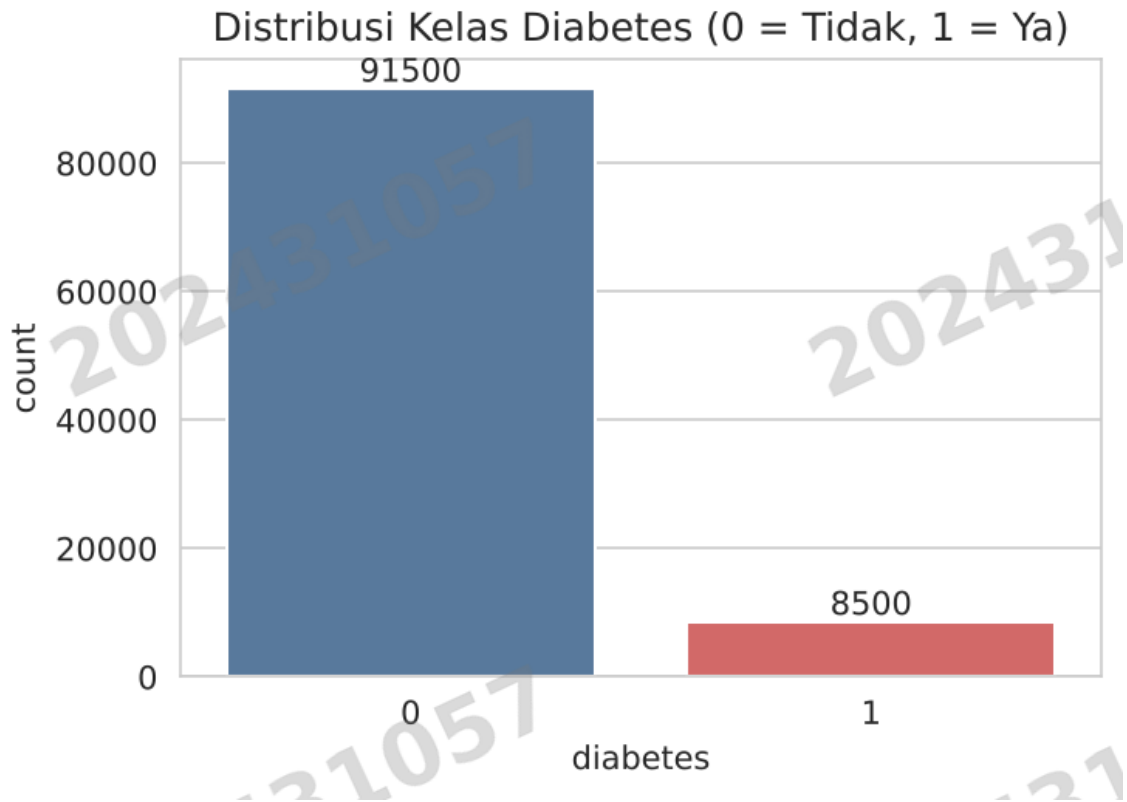
### 3.2.4 Cell 4: EDA: distribusi variabel target

```
In [4]: #202431057_HANA MALIDHA HASAN
        cnt = df["diabetes"].value_counts()
        print(cnt.to_string()); print((cnt / len(df) * 100).round(2).to_string())
        fig, ax = plt.subplots(figsize=(5, 3.6))
        sns.countplot(x="diabetes", data=df, palette=["#4C78A8", "#E45756"], ax=ax)
        for p in ax.patches:
            ax.annotate(int(p.get_height()), (p.get_x() + p.get_width() / 2, p.get_height()),
                        ha="center", va="bottom")
        ax.set_title("Distribusi Kelas Diabetes (0 = Tidak, 1 = Ya)")
        plt.tight_layout(); plt.show()
```

*Gambar 3.7 Kode Cell 4 (watermark NIM)*

```
Out[4]: diabetes
        0    91500
        1     8500
        diabetes
        0    91.5
        1     8.5
```

*Gambar 3.8 Output teks Cell 4*




*Gambar 3.9 Output grafik Cell 4*

**Kode.** Distribusi target dihitung dengan *value_counts()* lalu divisualisasikan dalam *countplot*, lengkap dengan label jumlah di atas batang.

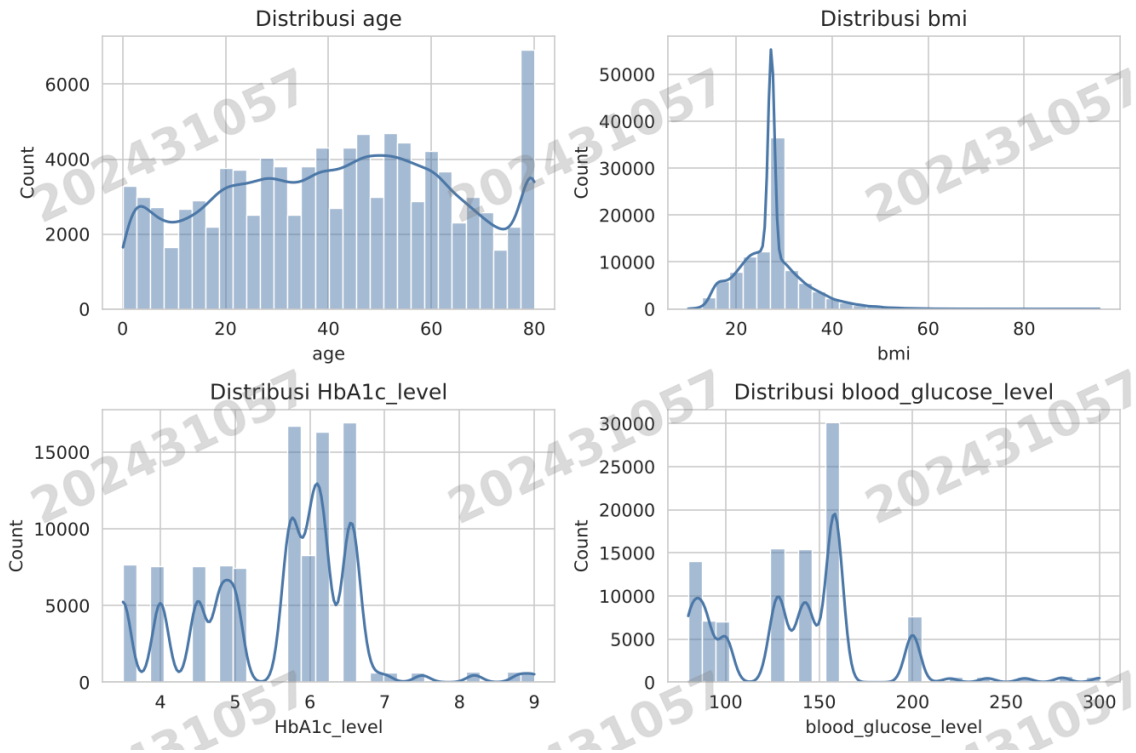
**Hasil.** Terdapat 91.500 data tidak diabetes (91,5%) dan 8.500 data diabetes (8,5%), atau rasio sekitar 10,8 : 1. Dataset ini jelas tidak seimbang. Konsekuensinya, model yang selalu menebak 'tidak

diabetes' sudah memperoleh akurasi 91,5%, sehingga akurasi harus dibaca bersama *recall* dan *F1-score* kelas diabetes.

### 3.2.5 Cell 5: EDA: Distribusi variabel numerik

```
In [5]: #202431057_HANA MALIDHA HASAN
        num_cols = ["age", "bmi", "HbA1c_level", "blood_glucose_level"]
        fig, axes = plt.subplots(2, 2, figsize=(9, 6))
        for ax, c in zip(axes.ravel(), num_cols):
            sns.histplot(df[c], bins=30, kde=True, color="#4C78A8", ax=ax)
            ax.set_title(f"Distribusi {c}")
        plt.tight_layout(); plt.show()
```

*Gambar 3.10 Kode Cell 5 (watermark NIM)*




*Gambar 3.11 Output grafik Cell 5*

**Kode.** Histogram disertai kurva *kernel density* dibuat untuk empat fitur numerik (*age*, *bmi*, *HbA1c_level*, *blood_glucose_level*) guna melihat bentuk sebaran.

**Hasil.** Usia tersebar cukup merata dengan lonjakan pada 80 tahun karena data dibatasi sampai 80. BMI condong ke kanan dengan puncak sangat tajam di sekitar 27,3; banyak observasi bernilai identik pada angka tersebut, yang sama dengan median dan rata-rata. HbA1c dan glukosa darah berbentuk multimodal dan diskret: nilainya menumpuk pada angka-angka tertentu (misalnya glukosa pada 80, 100, 126, 140, 158, dan 200) dengan ekor panjang ke nilai tinggi. Sebaran yang tidak normal ini bukan masalah bagi KNN, tetapi perlu diingat saat membaca asumsi Regresi Linear.

### 3.2.6 Cell 6: EDA: boxplot fitur numerik terhadap diabetes dan deteksi outlier

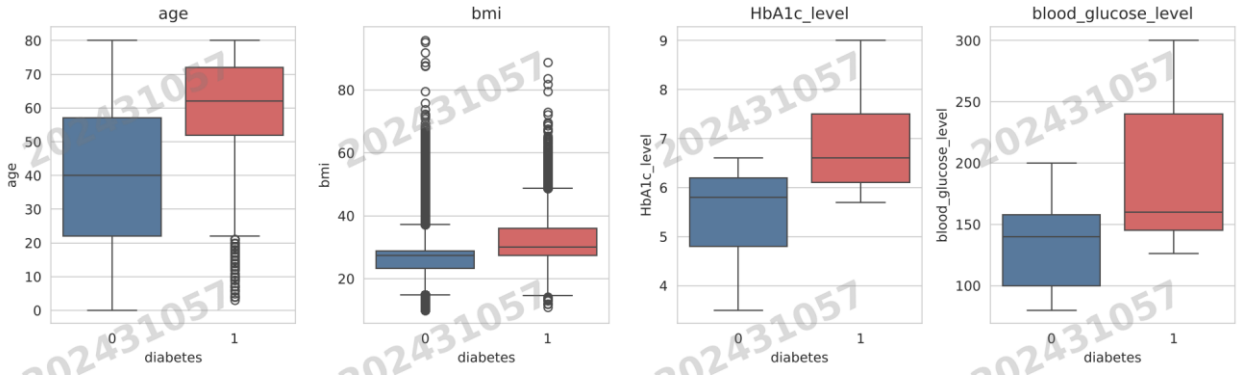
```
In [6]: #202431057_HANA MALIDHA HASAN
        fig, axes = plt.subplots(1, 4, figsize=(13, 4))
        for ax, c in zip(axes, num_cols):
            sns.boxplot(x="diabetes", y=c, data=df, palette=["#4C78A8", "#E45756"], ax=ax)
            ax.set_title(c)
        plt.tight_layout(); plt.show()

        for c in num_cols:
            q1, q3 = df[c].quantile([.25, .75]); iqr = q3 - q1
            n = ((df[c] < q1 - 1.5 * iqr) | (df[c] > q3 + 1.5 * iqr)).sum()
            print(f"{c:22s} outlier (IQR): {n:6d} ({n/len(df)*100:.2f}%)")
```

*Gambar 3.12 Kode Cell 6 (watermark NIM)*

```
Out[6]: age                  outlier (IQR):      0 (0.00%)
        bmi                  outlier (IQR):   7086 (7.09%)
        HbA1c_level          outlier (IQR):   1315 (1.31%)
        blood_glucose_level  outlier (IQR):   2038 (2.04%)
```

*Gambar 3.13 Output teks Cell 6*




*Gambar 3.14 Output grafik Cell 6*

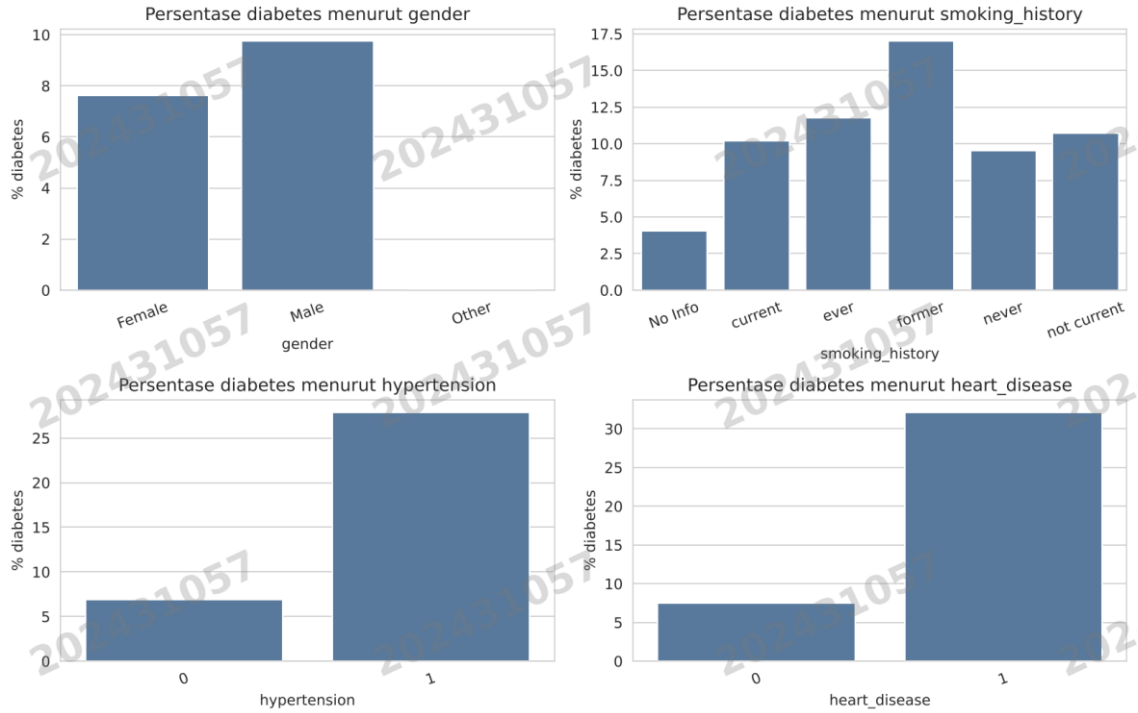
**Kode.** *Boxplot* membandingkan keempat fitur numerik antara kelompok tidak diabetes (0) dan diabetes (1). Selanjutnya *outlier* dihitung dengan aturan *IQR* (di luar Q1 − 1,5·IQR dan Q3 + 1,5·IQR).

**Hasil.** Kelompok diabetes memiliki median usia, BMI, HbA1c, dan glukosa darah yang lebih tinggi daripada kelompok tidak diabetes, dengan HbA1c dan glukosa menunjukkan pemisahan paling jelas. *Outlier* terdeteksi pada *bmi* sebanyak 7.086 baris (7,09%), *HbA1c_level* 1.315 baris (1,31%), dan *blood_glucose_level* 2.038 baris (2,04%), sedangkan *age* tidak memiliki *outlier*. *Outlier* tidak dibuang karena merupakan nilai klinis yang mungkin terjadi (obesitas berat, kadar gula sangat tinggi) dan justru menjadi ciri penderita diabetes. Membuangnya berisiko menghilangkan informasi penting.

### 3.2.7 Cell 7: EDA: fitur kategorikal terhadap proporsi diabetes

```
In [7]: #202431057_HANA MALIDHA HASAN
        cat_cols = ["gender", "smoking_history", "hypertension", "heart_disease"]
        fig, axes = plt.subplots(2, 2, figsize=(11, 7))
        for ax, c in zip(axes.ravel(), cat_cols):
            rate = df.groupby(c)["diabetes"].mean() * 100
            sns.barplot(x=rate.index.astype(str), y=rate.values, color="#4C78A8", ax=ax)
            ax.set_title(f"Persentase diabetes menurut {c}"); ax.set_ylabel("% diabetes")
            ax.tick_params(axis="x", rotation=20)
        plt.tight_layout(); plt.show()
```

*Gambar 3.15 Kode Cell 7 (watermark NIM)*




*Gambar 3.16 Output grafik Cell 7*

**Kode.** Persentase penderita diabetes dihitung per kategori dengan *groupby()* untuk fitur *gender*, *smoking_history*, *hypertension*, dan *heart_disease*, lalu ditampilkan dalam diagram batang.

**Hasil.** Persentase diabetes pada laki-laki (sekitar 9,7%) lebih tinggi daripada perempuan (sekitar 7,6%); kategori *Other* bernilai 0% namun hanya terdiri dari 18 baris. Perokok *former* memiliki persentase tertinggi (sekitar 17%), sedangkan *No Info* terendah (sekitar 4%). Pasien dengan hipertensi memiliki persentase diabetes sekitar 28% dibanding sekitar 7% tanpa hipertensi, dan pasien dengan penyakit jantung sekitar 32% dibanding sekitar 7%. Riwayat hipertensi dan penyakit jantung dengan demikian sangat berkaitan dengan diabetes.

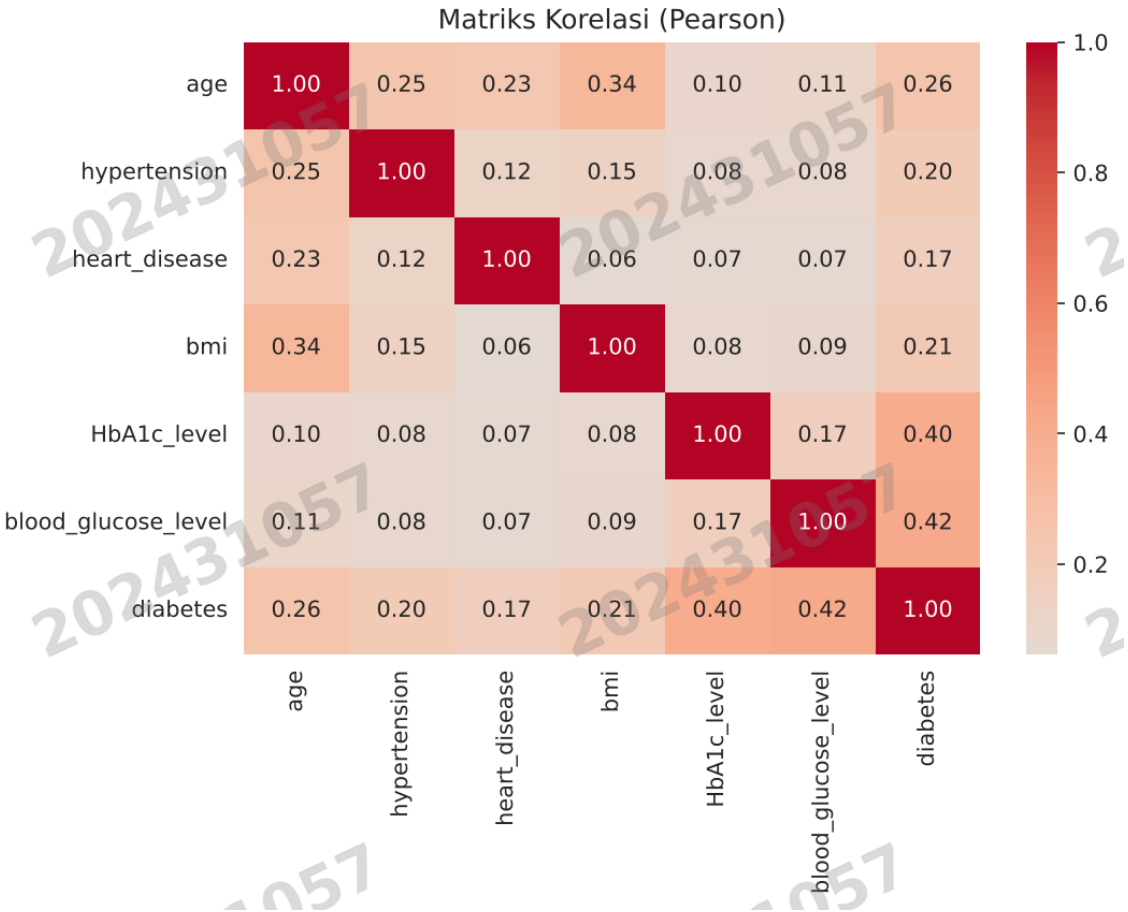
### 3.2.8 Cell 8: EDA: matriks korelasi

```
In [8]: #202431057_HANA MALIDHA HASAN
        corr = df.select_dtypes("number").corr()
        fig, ax = plt.subplots(figsize=(7.5, 5.8))
        sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0, ax=ax)
        ax.set_title("Matriks Korelasi (Pearson)")
        plt.tight_layout(); plt.show()
        print(corr["diabetes"].drop("diabetes").sort_values(ascending=False).round(3).to_string(
            ))
```

*Gambar 3.17 Kode Cell 8 (watermark NIM)*

```
Out[8]: blood_glucose_level    0.420
        HbA1c_level            0.401
        age                    0.258
        bmi                    0.214
        hypertension           0.198
        heart_disease          0.172
```

*Gambar 3.18 Output teks Cell 8*




*Gambar 3.19 Output grafik Cell 8*

**Kode.** Matriks korelasi *Pearson* dihitung untuk seluruh kolom numerik dan divisualisasikan sebagai *heatmap*. Korelasi setiap fitur terhadap target diurutkan dari yang terbesar.

**Hasil.** Semua fitur berkorelasi positif dengan diabetes: *blood_glucose_level* (0,420), *HbA1c_level* (0,401), *age* (0,258), *bmi* (0,214), *hypertension* (0,198), dan *heart_disease* (0,172).

Korelasi tertinggi hanya sedang, sehingga hubungan linear tunggal tidak cukup menjelaskan target. Korelasi antarfitur prediktor rendah hingga sedang (tertinggi *age–bmi* 0,34), sehingga tidak ada indikasi multikolinearitas yang mengganggu Regresi Linear. Jika korelasi dikuadratkan menjadi koefisien determinasi, $r^2$ tertinggi hanya 0,176 (glukosa darah) dan 0,161 (HbA1c), yang berdasarkan kategori pada Bab II tergolong sangat lemah untuk masing-masing fitur secara tunggal.

### 3.2.9 Cell 9: Preprocessing: hapus duplikat dan encoding kategorikal

```
In [9]: #202431057_HANA MALIDHA HASAN
        n_awal = len(df)
        df_clean = df.drop_duplicates().reset_index(drop=True)
        print(f"Baris sebelum: {n_awal} | sesudah: {len(df_clean)} | dihapus: {n_awal - len(df_c
              lean)}")

        # One-hot encoding untuk fitur kategorikal nominal (drop_first agar tidak multikolinear)
        df_enc = pd.get_dummies(df_clean, columns=["gender", "smoking_history"], drop_first=True
              , dtype=int)
        print("Bentuk data setelah encoding:", df_enc.shape)
        print("Distribusi target (%):"); print((df_enc["diabetes"].value_counts(normalize=True)
              * 100).round(2).to_string())
        df_enc.head()
```

*Gambar 3.20 Kode Cell 9 (watermark NIM)*

```
Baris sebelum: 100000 | sesudah: 96146 | dihapus: 3854
Bentuk data setelah encoding: (96146, 14)
Distribusi target (%):
0    91.18
1     8.82
```

Out[9]:

| | age | hypertension | heart_disease | bmi | HbA1c_level | blood_glucose_level | diabetes | gender_Male | gender_Other | smoking_history_current | smoking_history_ |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 80.0 | 0 | 1 | 25.19 | 6.6 | 140 | 0 | 0 | 0 | 0 | |
| 1 | 54.0 | 0 | 0 | 27.32 | 6.6 | 80 | 0 | 0 | 0 | 0 | |
| 2 | 28.0 | 0 | 0 | 27.32 | 5.7 | 158 | 0 | 1 | 0 | 0 | |
| 3 | 36.0 | 0 | 0 | 23.45 | 5.0 | 155 | 0 | 0 | 0 | 1 | |
| 4 | 76.0 | 1 | 1 | 20.14 | 4.8 | 155 | 0 | 1 | 0 | 1 | |

Out[9]:

| level | diabetes | gender_Male | gender_Other | smoking_history_current | smoking_history_ever | smoking_history_former | smoking_history_never | smoking_history_not current |
|---|---|---|---|---|---|---|---|---|
| 140 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 80 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 158 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 |
| 155 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 155 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 |

*Gambar 3.21 Output teks Cell 9*

**Kode.** Baris duplikat dihapus dengan *drop_duplicates()*. Kolom *gender* dan *smoking_history* kemudian diubah menjadi variabel biner dengan *pd.get_dummies(drop_first=True)* agar tidak terjadi jebakan variabel *dummy* (multikolinearitas sempurna).

**Hasil.** Jumlah baris turun dari 100.000 menjadi 96.146 (3.854 baris dihapus). Setelah *encoding* terdapat 14 kolom (13 fitur ditambah target). Proporsi kelas setelah duplikat dihapus menjadi 91,18% tidak diabetes dan 8,82% diabetes; ketidakseimbangan tetap ada sehingga penanganannya dibahas pada evaluasi model.

### 3.2.10 Cell 10: Preprocessing: pembagian data latih/uji dan standardisasi

```
In [10]: #202431057_HANA MALIDHA HASAN
         X = df_enc.drop(columns="diabetes")
         y = df_enc["diabetes"]
         X_train, X_test, y_train, y_test = train_test_split(
             X, y, test_size=0.2, random_state=SEED, stratify=y)

         scaler = StandardScaler()
         X_train_s = scaler.fit_transform(X_train)   # fit hanya pada data latih (hindari data le
             akage)
         X_test_s = scaler.transform(X_test)

         print("X_train:", X_train_s.shape, "| X_test:", X_test_s.shape)
         print("Proporsi diabetes train: %.3f | test: %.3f" % (y_train.mean(), y_test.mean()))
         print("Mean fitur (train, setelah scaling) ~", np.round(X_train_s.mean(), 4),
               "| Std ~", np.round(X_train_s.std(), 4))
```

*Gambar 3.22 Kode Cell 10 (watermark NIM)*

```
Out[10]: X_train: (76916, 13) | X_test: (19230, 13)
         Proporsi diabetes train: 0.088 | test: 0.088
         Mean fitur (train, setelah scaling) ~ -0.0 | Std ~ 1.0
```

*Gambar 3.23 Output teks Cell 10*

**Kode.** Data dibagi 80% latih dan 20% uji dengan *train_test_split* menggunakan *stratify=y* dan *random_state=42*. *StandardScaler* di-*fit* hanya pada data latih, lalu diterapkan pada data latih dan data uji.

**Hasil.** Data latih berukuran 76.916 × 13 dan data uji 19.230 × 13. Proporsi diabetes pada kedua bagian sama (0,088), sehingga pembagian *stratified* berhasil. Setelah standardisasi, rata-rata fitur latih mendekati 0 dan simpangan bakunya mendekati 1.

### 3.2.11 Cell 11: Model 1: Regresi Linear (pelatihan dan evaluasi regresi)

```
In [11]: #202431057_HANA MALIDHA HASAN
         lin = LinearRegression()
         lin.fit(X_train_s, y_train)
         y_pred_cont = lin.predict(X_test_s)          # keluaran kontinu (skor)

         print("== Metrik regresi pada data uji ==")
         print("MSE  :", round(mean_squared_error(y_test, y_pred_cont), 4))
         print("RMSE :", round(np.sqrt(mean_squared_error(y_test, y_pred_cont)), 4))
         print("MAE  :", round(mean_absolute_error(y_test, y_pred_cont), 4))
         print("R2   :", round(r2_score(y_test, y_pred_cont), 4))
         print("\nIntercept:", round(lin.intercept_, 4))
         coef = pd.Series(lin.coef_, index=X.columns).sort_values(key=abs, ascending=False)
         print(coef.round(4).to_string())
```

*Gambar 3.24 Kode Cell 11 (watermark NIM)*

```
Out[11] == Metrik regresi pada data uji ==
        MSE  : 0.0519
        RMSE : 0.2279
        MAE  : 0.1573
        R2   : 0.3542

        Intercept: 0.0882
        blood_glucose_level           0.0944
        HbA1c_level                   0.0891
        age                           0.0323
        bmi                           0.0268
        hypertension                  0.0252
        heart_disease                 0.0230
        gender_Male                   0.0076
        smoking_history_former        0.0065
        smoking_history_never         0.0038
        smoking_history_not current   0.0025
        smoking_history_current       0.0013
        smoking_history_ever          0.0011
        gender_Other                 -0.0005
```

*Gambar 3.25 Output teks Cell 11*

**Kode.** Model *LinearRegression* dilatih pada data latih yang sudah distandardisasi, kemudian memprediksi skor kontinu pada data uji. Metrik regresi (MSE, RMSE, MAE, $R^2$) dihitung, dan koefisien model diurutkan berdasarkan nilai mutlaknya.

**Hasil.** Pada data uji diperoleh MSE 0,0519, RMSE 0,2279, MAE 0,1573, dan $R^2$ 0,3542, artinya model hanya menjelaskan sekitar 35,4% variasi target (kategori lemah menurut klasifikasi pada Bab II). *Intercept* 0,0882 hampir sama dengan proporsi diabetes, sebagaimana diharapkan pada fitur yang berpusat di nol. Koefisien terbesar dimiliki *blood_glucose_level* (0,0944) dan *HbA1c_level* (0,0891), diikuti *age* (0,0323), *bmi* (0,0268), *hypertension* (0,0252), dan *heart_disease* (0,0230). Urutan ini konsisten dengan hasil korelasi pada EDA. Koefisien *smoking_history* dan *gender* sangat kecil, sehingga hampir tidak berkontribusi.

### 3.2.12 Cell 12: Model 1: Regresi Linear (konversi ke kelas dan akurasi)

```
In [12]:#202431057_HANA MALIDHA HASAN
        y_pred_lin = (y_pred_cont >= 0.5).astype(int)   # ambang keputusan 0,5

        acc_lin = accuracy_score(y_test, y_pred_lin)
        print(f"Akurasi Regresi Linear : {acc_lin:.4f} ({acc_lin*100:.2f}%)\n")
        print("Confusion matrix [[TN, FP], [FN, TP]]:"); print(confusion_matrix(y_test, y_pred_l
              in)); print()
        print("Baseline kelas mayoritas: %.4f" % (1 - y_test.mean()), "\n")
        print(classification_report(y_test, y_pred_lin, target_names=["Tidak diabetes", "Diabete
              s"], digits=4))

        fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))
        ConfusionMatrixDisplay(confusion_matrix(y_test, y_pred_lin),
            display_labels=["Tidak", "Diabetes"]).plot(ax=axes[0], cmap="Blues", colorbar=False)
        axes[0].set_title("Confusion Matrix - Regresi Linear")
        sns.histplot(x=y_pred_cont, hue=y_test.map({0: "Tidak", 1: "Diabetes"}), bins=40,
                     stat="count", ax=axes[1], palette=["#4C78A8", "#E45756"])
        axes[1].axvline(0.5, color="k", ls="--"); axes[1].set_title("Skor Prediksi vs Kelas Aktu
              al")
        plt.tight_layout(); plt.show()
```

*Gambar 3.26 Kode Cell 12 (watermark NIM)*

```
Out[12]:Akurasi Regresi Linear : 0.9396 (93.96%)

        Confusion matrix [[TN, FP], [FN, TP]]:
        [[17525     9]
         [ 1152   544]]

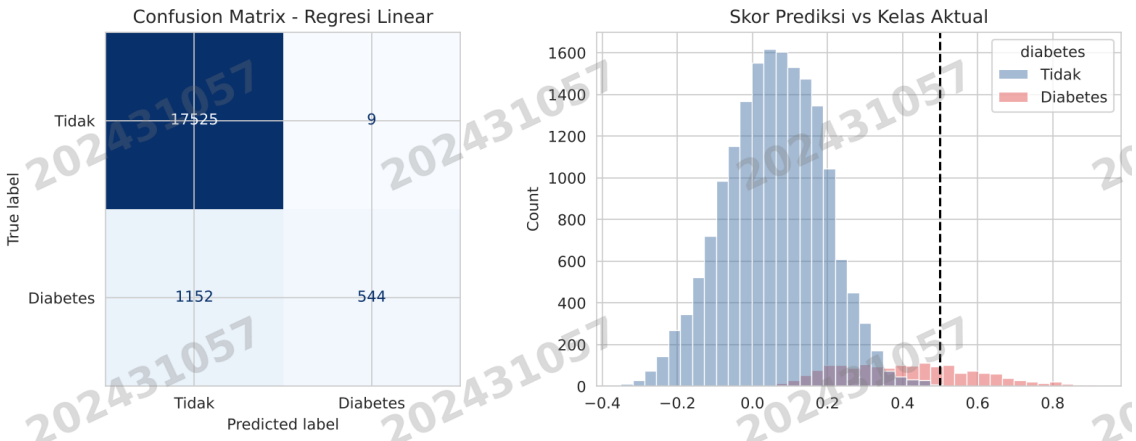
        Baseline kelas mayoritas: 0.9118

                      precision    recall  f1-score   support

        Tidak diabetes     0.9383    0.9995    0.9679     17534
             Diabetes      0.9837    0.3208    0.4838      1696

             accuracy                         0.9396     19230
            macro avg      0.9610    0.6601    0.7259     19230
         weighted avg      0.9423    0.9396    0.9252     19230
```

*Gambar 3.27 Output teks Cell 12*




*Gambar 3.28 Output grafik Cell 12*

**Kode.** Skor kontinu diubah menjadi kelas dengan ambang 0,5 (>= *0.5* berarti diabetes). Kemudian dihitung akurasi, matriks konfusi, *baseline* kelas mayoritas, dan *classification report*. Grafik kedua menunjukkan sebaran skor prediksi untuk tiap kelas aktual.

**Hasil.** Akurasi Regresi Linear sebesar **0,9396 (93,96%)**. Matriks konfusi: TN = 17,525, FP = 9, FN = 1,152, TP = 544. *Precision* kelas diabetes tinggi (0,9837) tetapi *recall* hanya 0,3208, artinya model hanya menemukan sekitar sepertiga penderita; *F1-score* kelas diabetes 0,4838. Akurasi *baseline* kelas mayoritas adalah 0.9118, sehingga keunggulan model hanya sekitar 2.78 poin persentase. Histogram skor menunjukkan kedua kelas saling tumpang tindih, dan sebagian skor bernilai negatif (hingga sekitar −0,4), yang menegaskan keterbatasan Regresi Linear sebagai pengklasifikasi. Ambang 0,5 membuat model sangat konservatif terhadap kelas minoritas, sehingga akurasi yang tinggi lebih banyak berasal dari kelas mayoritas.

### 3.2.13 Cell 13: Model 2: KNN baseline (k = 5)

```
In [13]: #202431057_HANA MALIDHA HASAN
         knn0 = KNeighborsClassifier(n_neighbors=5)
         knn0.fit(X_train_s, y_train)
         y_pred_knn0 = knn0.predict(X_test_s)
         print(f"Akurasi KNN baseline (k=5): {accuracy_score(y_test, y_pred_knn0):.4f}")
         print(f"Recall kelas diabetes     : {recall_score(y_test, y_pred_knn0):.4f}")
```

*Gambar 3.29 Kode Cell 13 (watermark NIM)*

```
Out[13]: Akurasi KNN baseline (k=5): 0.9591
         Recall kelas diabetes     : 0.6162
```

*Gambar 3.30 Output teks Cell 13*

**Kode.** Sebagai pembanding awal, *KNeighborsClassifier* dengan *n_neighbors=5* (nilai bawaan yang umum) dilatih dan diuji tanpa *tuning*.

**Hasil.** Akurasi *baseline* KNN sebesar 0,9591 dengan *recall* kelas diabetes 0,6162. Bahkan tanpa *tuning*, KNN sudah mengungguli Regresi Linear baik pada akurasi maupun *recall*, karena KNN dapat menangkap hubungan nonlinear antarfitur.

### 3.2.14 Cell 14: Model 2: Tuning hyperparameter KNN dengan GridSearchCV

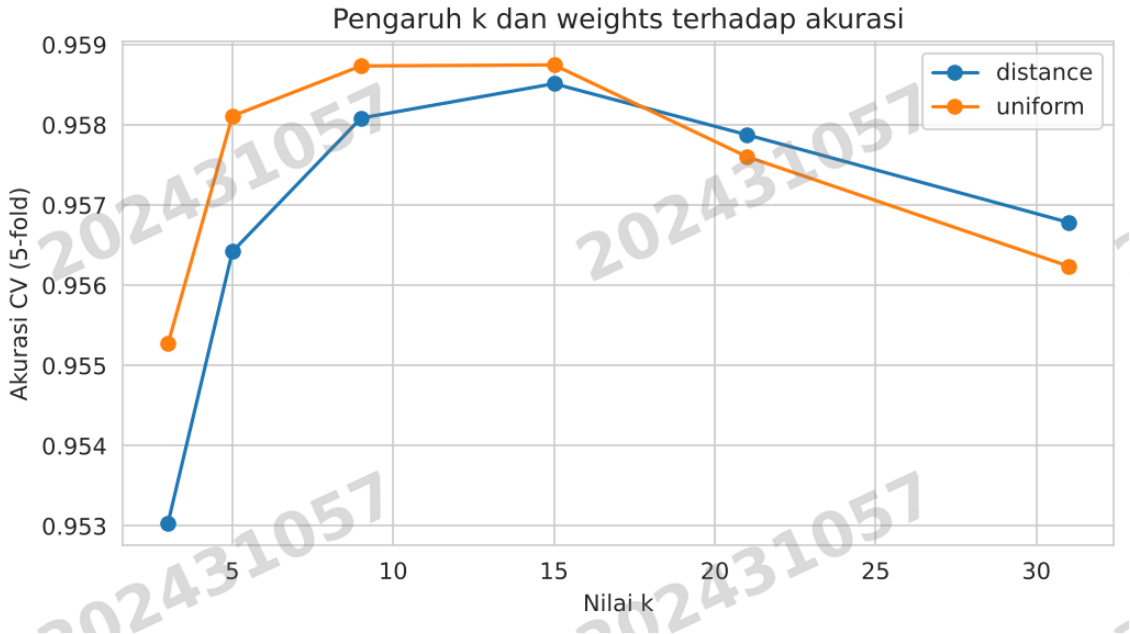
```
In [14]:#202431057_HANA MALIDHA HASAN
param_grid = {"n_neighbors": [3, 5, 9, 15, 21, 31],
              "weights": ["uniform", "distance"]}
gs = GridSearchCV(KNeighborsClassifier(), param_grid, cv=StratifiedKFold(5, shuffle=True
      , random_state=SEED),
                  scoring="accuracy", n_jobs=2)
gs.fit(X_train_s, y_train)
print("Parameter terbaik :", gs.best_params_)
print(f"Akurasi CV terbaik: {gs.best_score_:.4f}")

res = pd.DataFrame(gs.cv_results_)
print(res[["param_n_neighbors", "param_weights", "mean_test_score"]].sort_values("mean_t
      est_score", ascending=False).head(6).round(4).to_string(index=False))
fig, ax = plt.subplots(figsize=(7, 4))
for w, g in res.groupby("param_weights"):
    ax.plot(g["param_n_neighbors"].astype(int), g["mean_test_score"], marker="o", label=
      w)
ax.set_xlabel("Nilai k"); ax.set_ylabel("Akurasi CV (5-fold)")
ax.set_title("Pengaruh k dan weights terhadap akurasi"); ax.legend()
plt.tight_layout(); plt.show()
```

*Gambar 3.31 Kode Cell 14 (watermark NIM)*

```
Out[14]:Parameter terbaik : {'n_neighbors': 15, 'weights': 'uniform'}
Akurasi CV terbaik: 0.9587
 param_n_neighbors param_weights  mean_test_score
                15       uniform           0.9587
                 9       uniform           0.9587
                15      distance           0.9585
                 5       uniform           0.9581
                 9      distance           0.9581
                21      distance           0.9579
```

*Gambar 3.32 Output teks Cell 14*




*Gambar 3.33 Output grafik Cell 14*

**Kode.** *GridSearchCV* mencari kombinasi terbaik *n_neighbors* ∈ {3, 5, 9, 15, 21, 31} dan *weights* ∈ {*uniform*, *distance*} dengan *stratified 5-fold cross-validation* pada data latih (6 × 2 × 5 =

60 pelatihan). Skor yang dioptimalkan adalah akurasi. Hasil *cross-validation* divisualisasikan sebagai kurva *k* terhadap akurasi.

**Hasil.** Parameter terbaik adalah **n_neighbors = 15 dengan weights = uniform** (akurasi *cross-validation* 0,9587). Kurva memperlihatkan pola khas *bias–variance*: akurasi naik dari *k* = 3 hingga puncak di sekitar *k* = 9–15 lalu turun perlahan pada *k* yang lebih besar karena model terlalu halus. Pada *k* kecil model terlalu peka terhadap *noise*. Perbedaan akurasi antarkombinasi relatif kecil (sekitar 0,953–0,959), sehingga KNN pada dataset ini cukup stabil terhadap pemilihan *k*. Nilai *k* = 9 dan *k* = 15 menghasilkan akurasi *cross-validation* yang sama (0,9587). Pembobotan *uniform* sedikit lebih baik daripada *distance* pada hampir semua nilai *k*.

### 3.2.15 Cell 15: Model 2: Evaluasi KNN terbaik pada data uji

```
In [15]: #202431057_HANA MALIDHA HASAN
         knn = gs.best_estimator_
         y_pred_knn = knn.predict(X_test_s)
         acc_knn = accuracy_score(y_test, y_pred_knn)
         print(f"Akurasi KNN (tuned) : {acc_knn:.4f} ({acc_knn*100:.2f}%)\n")
         print("Confusion matrix [[TN, FP], [FN, TP]]:"); print(confusion_matrix(y_test, y_pred_k
               nn)); print()
         print(classification_report(y_test, y_pred_knn, target_names=["Tidak diabetes", "Diabete
               s"], digits=4))

         fig, ax = plt.subplots(figsize=(4.8, 4.2))
         ConfusionMatrixDisplay(confusion_matrix(y_test, y_pred_knn),
             display_labels=["Tidak", "Diabetes"]).plot(ax=ax, cmap="Greens", colorbar=False)
         ax.set_title("Confusion Matrix - KNN")
         plt.tight_layout(); plt.show()
```

*Gambar 3.34 Kode Cell 15 (watermark NIM)*

```
Out[15]: Akurasi KNN (tuned) : 0.9603 (96.03%)

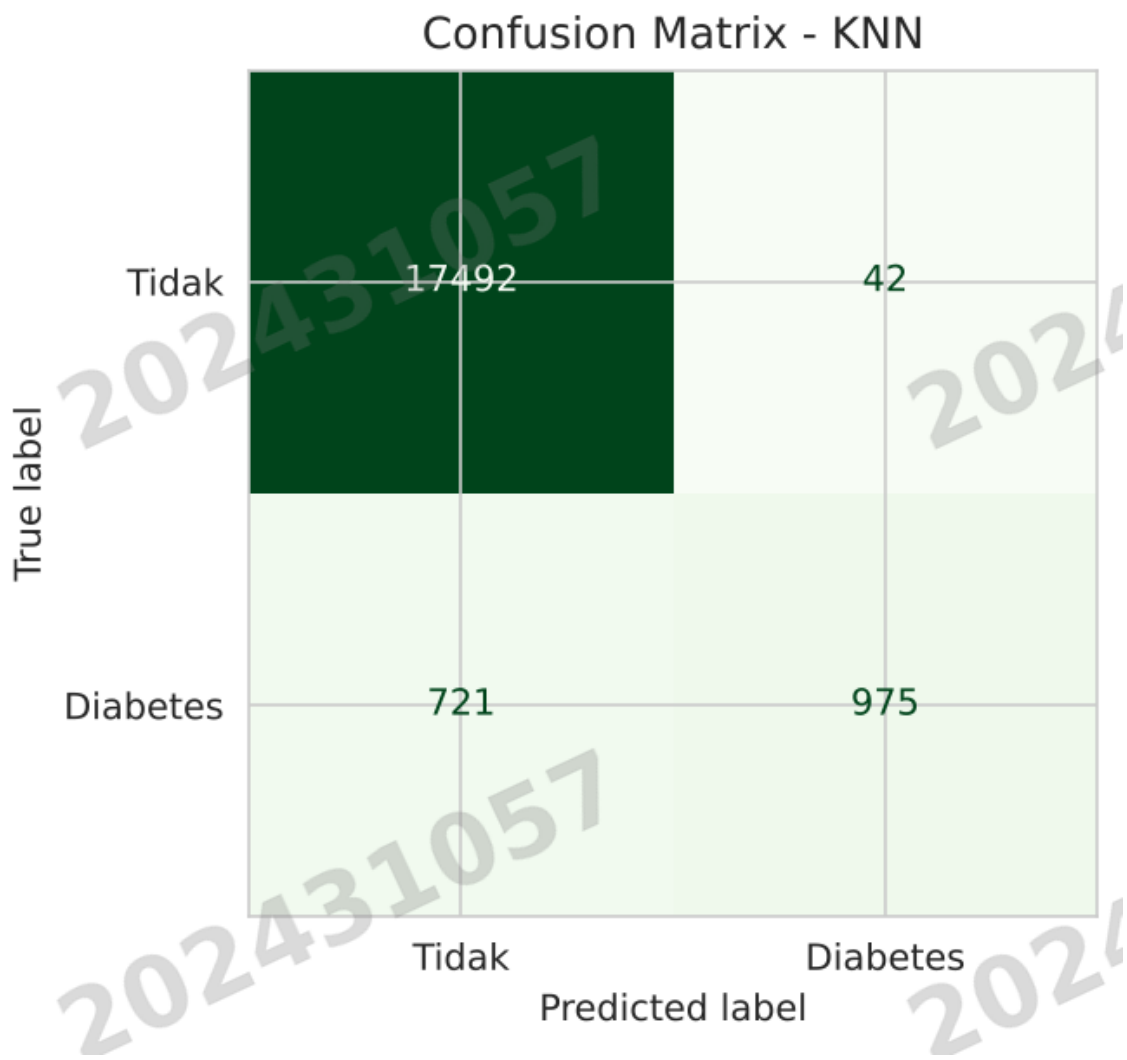
         Confusion matrix [[TN, FP], [FN, TP]]:
         [[17492    42]
          [  721   975]]

                       precision    recall  f1-score   support

         Tidak diabetes     0.9604    0.9976    0.9787     17534
              Diabetes      0.9587    0.5749    0.7188      1696

               accuracy                         0.9603     19230
              macro avg      0.9596    0.7862    0.8487     19230
           weighted avg      0.9603    0.9603    0.9557     19230
```

*Gambar 3.35 Output teks Cell 15*




*Gambar 3.36 Output grafik Cell 15*

**Kode.** Model terbaik dari *GridSearchCV* (*best_estimator_*) dievaluasi pada data uji yang belum pernah dipakai selama *tuning*, dengan akurasi, matriks konfusi, dan *classification report*.

**Hasil.** Akurasi KNN terbaik pada data uji adalah **0,9603 (96,03%)**. Matriks konfusi: TN = 17,492, FP = 42, FN = 721, TP = 975. *Precision* kelas diabetes 0,9587, *recall* 0,5749, dan *F1-score* 0,7188. Dibanding Regresi Linear, KNN menemukan 431 penderita lebih banyak (TP 975 berbanding 544) dengan tambahan 33 *false positive*. Akurasi data uji (0,9603) sangat dekat dengan akurasi *cross-validation* (0,9587), yang menandakan model tidak *overfitting*. Meski demikian, hampir 43% penderita masih terlewat (FN = 721).

### 3.2.16 Cell 16: Model 2: KNN dengan k sesuai nomor PC (k = 9)

```
In [16]: #202431057_HANA MALIDHA HASAN
         K_TARGET = 9   # Nomor PC praktikum
         knn_pc = KNeighborsClassifier(n_neighbors=K_TARGET)
         knn_pc.fit(X_train_s, y_train)
         y_pred_pc = knn_pc.predict(X_test_s)
         print(f"Nilai K yang digunakan : {K_TARGET}")
         print(f"Akurasi KNN (k = {K_TARGET})   : {accuracy_score(y_test, y_pred_pc):.4f}\n")
         print("Confusion matrix [[TN, FP], [FN, TP]]:"); print(confusion_matrix(y_test, y_pred_p
             c)); print()
         print(classification_report(y_test, y_pred_pc, target_names=["Tidak diabetes", "Diabetes
               "], digits=4))
```

*Gambar 3.37 Kode Cell 16 (watermark NIM)*

```
Out[16]: Nilai K yang digunakan : 9
         Akurasi KNN (k = 9)   : 0.9595

         Confusion matrix [[TN, FP], [FN, TP]]:
         [[17452    82]
          [  696  1000]]

                       precision    recall  f1-score   support

         Tidak diabetes     0.9616    0.9953    0.9782     17534
              Diabetes     0.9242    0.5896    0.7199      1696

               accuracy                         0.9595     19230
              macro avg     0.9429    0.7925    0.8491     19230
           weighted avg     0.9583    0.9595    0.9554     19230
```

*Gambar 3.38 Output teks Cell 16*

**Kode.** Sesuai ketentuan praktikum, nilai *k* ditetapkan sama dengan nomor PC, yaitu **9** (*K_TARGET = 9*). Model *KNeighborsClassifier* dengan *k* = 9 dilatih pada data latih yang sudah distandardisasi, lalu dievaluasi pada data uji.

**Hasil.** Akurasi KNN dengan *k* = 9 adalah **0,9595 (95,95%)**. Matriks konfusi: TN = 17,452, FP = 82, FN = 696, TP = 1,000. *Precision* kelas diabetes 0,9242, *recall* 0,5896, dan *F1-score* 0,7199. Akurasinya hanya 0,08 poin persentase di bawah KNN hasil *tuning* (*k* = 15), sejalan dengan kurva *cross-validation* yang menunjukkan *k* = 9 dan *k* = 15 memiliki akurasi yang sama (0,9587). Dengan *k* lebih kecil, model sedikit lebih sensitif terhadap kelas diabetes: *recall* naik dari 0,5749 menjadi 0,5896, dengan harga *false positive* naik dari 42 menjadi 82.

### 3.2.17 Cell 17: Perbandingan Regresi Linear vs KNN

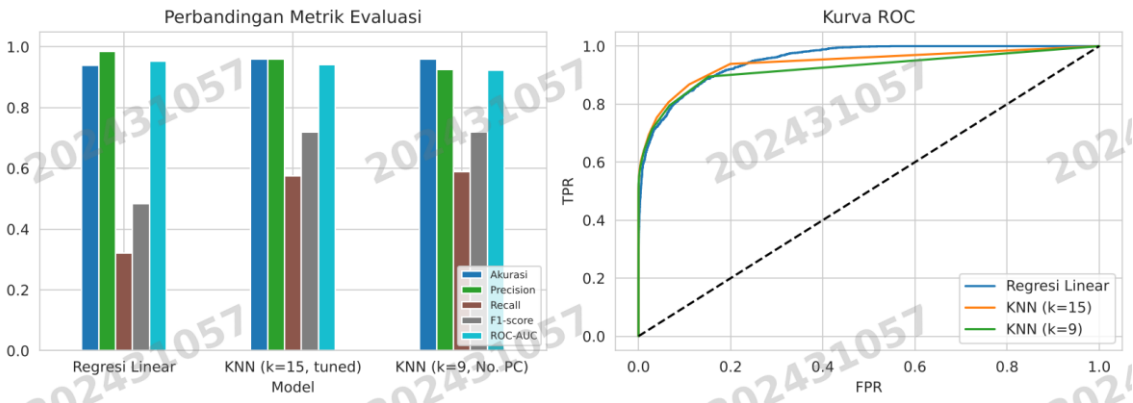
```
In [17]: #202431057_HANA MALIDHA HASAN
         proba_knn = knn.predict_proba(X_test_s)[:, 1]
         rows = []
         proba_pc = knn_pc.predict_proba(X_test_s)[:, 1]
         for name, yp, score in [("Regresi Linear", y_pred_lin, y_pred_cont), ("KNN (k=15, tuned)
               ", y_pred_knn, proba_knn), ("KNN (k=9, No. PC)", y_pred_pc, proba_pc)]:
             rows.append({"Model": name,
                          "Akurasi": accuracy_score(y_test, yp),
                          "Precision": precision_score(y_test, yp),
                          "Recall": recall_score(y_test, yp),
                          "F1-score": f1_score(y_test, yp),
                          "ROC-AUC": roc_auc_score(y_test, score)})
         hasil = pd.DataFrame(rows).set_index("Model").round(4)
         print(hasil.to_string())

         fig, axes = plt.subplots(1, 2, figsize=(11, 4))
         hasil.plot(kind="bar", ax=axes[0], rot=0, colormap="tab10"); axes[0].set_ylim(0, 1.05)
         axes[0].set_title("Perbandingan Metrik Evaluasi"); axes[0].legend(fontsize=7, loc="lower
                right")
         for name, score in [("Regresi Linear", y_pred_cont), ("KNN (k=15)", proba_knn), ("KNN (k
               =9)", proba_pc)]:
             fpr, tpr, _ = roc_curve(y_test, score); axes[1].plot(fpr, tpr, label=name)
         axes[1].plot([0, 1], [0, 1], "k--"); axes[1].set_xlabel("FPR"); axes[1].set_ylabel("TPR"
               )
         axes[1].set_title("Kurva ROC"); axes[1].legend()
         plt.tight_layout(); plt.show()
```

*Gambar 3.39 Kode Cell 17 (watermark NIM)*

```
Out[17]:                  Akurasi  Precision  Recall  F1-score  ROC-AUC
         Model
         Regresi Linear      0.9396     0.9837  0.3208    0.4838   0.9518
         KNN (k=15, tuned)   0.9603     0.9587  0.5749    0.7188   0.9409
         KNN (k=9, No. PC)   0.9595     0.9242  0.5896    0.7199   0.9226
```

*Gambar 3.40 Output teks Cell 17*




*Gambar 3.41 Output grafik Cell 17*

**Kode.** Metrik ketiga model dikumpulkan dalam satu tabel (akurasi, *precision*, *recall*, *F1-score*, ROC-AUC) lalu divisualisasikan dengan diagram batang dan kurva ROC. Untuk Regresi Linear skor kontinu dipakai sebagai skor ROC, sedangkan KNN memakai probabilitas kelas.

**Hasil.** Kedua varian KNN unggul atas Regresi Linear pada akurasi (0,9603 dan 0,9595 vs 0,9396), *recall* (0,5749 dan 0,5896 vs 0,3208), dan *F1-score* (0,7188 dan 0,7199 vs 0,4838). Regresi

Linear unggul pada *precision* (0,9837) dan ROC-AUC (0,9518). Kedua varian KNN nyaris setara: $k$ = 9 sedikit lebih baik pada *recall* dan *F1-score*, sedangkan $k$ = 15 lebih baik pada akurasi, *precision*, dan ROC-AUC. Kurva ROC KNN bergerigi karena probabilitasnya diskret (kelipatan 1/k).

### 3.3 Pembahasan Hasil

Ringkasan kinerja kedua algoritma pada data uji (19.230 sampel; 1.696 di antaranya penderita diabetes) ditunjukkan pada Tabel 3.1. Selain KNN hasil *tuning* ($k$ = 15), ditampilkan juga KNN dengan $k$ = 9 sesuai nomor PC.

*Tabel 3.1 Perbandingan kinerja Regresi Linear dan KNN pada data uji*

| Model | Akurasi | Precision | Recall | F1-score | ROC-AUC |
|---|---|---|---|---|---|
| Regresi Linear (ambang 0,5) | 0,9396 | 0,9837 | 0,3208 | 0,4838 | 0,9518 |
| KNN (k = 15, hasil tuning) | 0,9603 | 0,9587 | 0,5749 | 0,7188 | 0,9409 |
| KNN (k = 9, nomor PC) | 0,9595 | 0,9242 | 0,5896 | 0,7199 | 0,9226 |
| Baseline kelas mayoritas | 0,9118 | - | 0 | - | 0,5 |

**Akurasi.** KNN memperoleh akurasi 96,03% ($k$ = 15) dan 95,95% ($k$ = 9), lebih tinggi 2,07 dan 1,99 poin persentase daripada Regresi Linear (93,96%). Namun kedua nilai harus dibandingkan dengan *baseline* kelas mayoritas sebesar 91,18%: Regresi Linear hanya melampaui *baseline* sekitar 2,78 poin, sedangkan KNN sekitar 4,85 poin. Akurasi yang tinggi pada kedua model karenanya sebagian besar berasal dari kelas mayoritas.

**Penyebab perbedaan.** Regresi Linear mengasumsikan hubungan linear antara fitur dan target, sedangkan hubungan sebenarnya antara usia, BMI, HbA1c, dan glukosa dengan diabetes bersifat nonlinear dan melibatkan ambang (misalnya HbA1c yang naik tajam di atas sekitar 6,5). Model yang hanya mampu menarik batas linear dengan target biner menghasilkan skor yang tumpang tindih antarkelas ($R^2$ hanya 0,3542), sehingga dengan ambang 0,5 hanya kasus yang sangat jelas yang terdeteksi: *precision* tinggi tetapi *recall* rendah. KNN bersifat lokal dan nonparametrik sehingga dapat mengikuti batas keputusan yang tidak linear, yang tercermin pada *recall* dan *F1-score* yang jauh lebih baik. Hasil ini sejalan dengan temuan bahwa KNN efektif sebagai pengklasifikasi sederhana asalkan $k$, metrik jarak, dan penskalaan fitur dipilih dengan baik (Halder et al., 2024).

**ROC-AUC.** Regresi Linear justru sedikit lebih tinggi pada ROC-AUC (0,9518 vs 0,9409 dan 0,9226). Artinya kemampuan Regresi Linear dalam mengurutkan pasien berdasarkan risiko cukup baik; kelemahannya terletak pada ambang 0,5 yang tidak cocok untuk data tidak seimbang. Menurunkan ambang keputusan, memberi bobot pada kelas minoritas, atau *resampling* berpotensi memperbaiki *recall* kedua model (Sadeghi et al., 2022). Hal tersebut tidak dilakukan di sini agar perbandingan tetap berada pada pengaturan standar yang diminta soal.

**Keterbatasan.** (1) *Recall* kelas diabetes pada kedua model masih rendah (0,32 dan 0,57), padahal pada skrining penyakit kasus yang terlewat (*false negative*) lebih berbahaya daripada *false positive*. (2) Ambang 0,5 dan pembobotan *uniform* tidak dioptimalkan untuk *F1-score*. (3) Pembagian data hanya dilakukan satu kali dengan *random_state* 42, sehingga variasi antarpembagian tidak diukur. (4) Dataset berisi nilai yang banyak berulang (misalnya BMI 27,32), yang menunjukkan sebagian data kemungkinan hasil imputasi atau sintetis, sehingga hasil tidak otomatis dapat digeneralisasi ke data klinis nyata.

# BAB IV
# PENUTUP

## Kesimpulan

1. EDA menunjukkan dataset diabetes berukuran 100.000 baris tanpa *missing value* tetapi memiliki 3.854 baris duplikat dan kelas yang tidak seimbang (8,5% diabetes). Glukosa darah dan HbA1c adalah fitur dengan hubungan terkuat terhadap diabetes (korelasi 0,420 dan 0,401), diikuti usia, BMI, hipertensi, dan penyakit jantung.
2. *Preprocessing* yang dilakukan (penghapusan duplikat, *one-hot encoding*, pembagian *stratified* 80:20, dan standardisasi yang di-*fit* hanya pada data latih) menghasilkan 96.146 baris dengan 13 fitur siap dilatih.
3. Regresi Linear menghasilkan akurasi **93,96%** ($R^2$ 0,3542; RMSE 0,2279) dengan *recall* kelas diabetes 0,3208 dan *F1-score* 0,4838, sedangkan KNN dengan $k$ = 15 dan *weights uniform* hasil *GridSearchCV* menghasilkan akurasi **96,03%** dengan *recall* 0,5749 dan *F1-score* 0,7188. KNN dengan $k$ = 9 sesuai nomor PC menghasilkan akurasi **95,95%** (*recall* 0,5896; *F1-score* 0,7199), hampir setara dengan hasil *tuning*.
4. KNN lebih sesuai untuk dataset ini karena mampu memodelkan hubungan nonlinear. Meski begitu, kedua model hanya sedikit di atas akurasi *baseline* 91,18% dan masih melewatkan banyak penderita, sehingga akurasi tidak boleh dijadikan satu-satunya ukuran.

## Saran

Penelitian lanjutan dapat menambahkan penyesuaian ambang keputusan atau pembobotan kelas untuk meningkatkan *recall*, mencoba model yang dirancang untuk klasifikasi seperti regresi logistik atau *decision tree*, menggunakan *cross-validation* pada seluruh evaluasi, serta mengoptimalkan *F1-score* atau ROC-AUC alih-alih akurasi saat *tuning*.

# DAFTAR PUSTAKA

Arora, I., Khanduja, N., & Bansal, M. (2022). Effect of distance metric and feature scaling on KNN algorithm while classifying X-rays. In *Proceedings of the 10th seminary of computer science research at feminine (RIF'21)* (CEUR Workshop Proceedings, Vol. 3176). CEUR-WS.org. https://ceur-ws.org/Vol-3176/paper5.pdf

Chicco, D., Warrens, M. J., & Jurman, G. (2021). The coefficient of determination R-squared is more informative than SMAPE, MAE, MAPE, MSE and RMSE in regression analysis evaluation. *PeerJ Computer Science*, *7*, Article e623. https://doi.org/10.7717/peerj-cs.623

Halder, R. K., Uddin, M. N., Uddin, M. A., Aryal, S., & Khraisat, A. (2024). Enhancing K-nearest neighbor algorithm: A comprehensive review and performance analysis of modifications. *Journal of Big Data*, *11*, Article 113. https://doi.org/10.1186/s40537-024-00973-y

Hodson, T. O. (2022). Root mean square error (RMSE) or mean absolute error (MAE): When to use them or not. *Geoscientific Model Development*, *15*(14), 5481–5487. https://doi.org/10.5194/gmd-15-5481-2022

Sadeghi, S., Khalili, D., Ramezankhani, A., Mansournia, M. A., & Parsaeian, M. (2022). Diabetes mellitus risk prediction in the presence of class imbalance using flexible machine learning methods. *BMC Medical Informatics and Decision Making*, *22*. https://doi.org/10.1186/s12911-022-01775-z

Ukey, N., Yang, Z., Li, B., Zhang, G., Hu, Y., & Zhang, W. (2023). Survey on exact kNN queries over high-dimensional data space. *Sensors*, *23*(2), Article 629. https://doi.org/10.3390/s23020629